Searching and Sorting

Computer Programming 1

17/10/2026

Example: Search in non-ordered array



./cpp/linear_search.cpp

Example: Search in non-ordered array (with break)

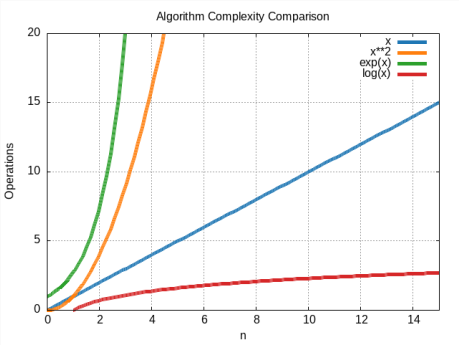


`./cpp/linear_search_break.cpp`

Searching in an Array

How many steps could require to search an element in an array?

- With a generic array:
  - $\approx N/2$ if the element is present,
    $\approx N$ otherwise
  - $O(N)$ (number proportional to N)
  - E.g., $N = 1,000,000 \rightarrow 1,000,000$ steps
- With an ordered array:
  - $\leq \lceil \log_2(N) \rceil$ steps
  - $O(\log_2(N))$
  - E.g., $N = 1,000,000 \rightarrow 20$ steps



Example: Searching in ordered array



`./cpp/binary_search.cpp`

Ordering Methods Overview



- Several ordering methods exist:
  - Selection sort
  - Insertion sort
  - Bubble sort
  - Quick sort
  - Shell sort
- Q. Why not only one?
- A. Some are simpler to understand + some are better than others in specific conditions (for instance: array nearly sorted, cost of swap operations)

Selection Sort



- Concept: Look for the smallest element and exchange it with the first element; repeat for the second smallest, etc., until ordered.
- Complexity: $O(N^2)$ steps on average.

Selection Sort: Implementation

```
void selectionsort(int A[], int N) {
  for (int i = 0; i < N - 1; i++) {
    int min = i;

    for (int j = i + 1; j < N; j++)
      if (A[j] < A[min]) min = j;

    swap(A[i], A[min]);
  }
}
```

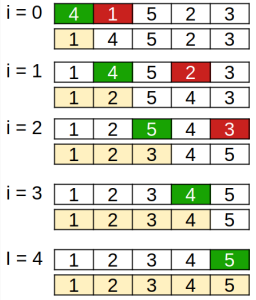
- outer loop takes care of ensuring the i-th element in the array is sorted
- inner loop selects the lowest element from the i-th position and swaps it

Selection Sort: Example

```
void selectionsort(int A[], int N) {
  for (int i = 0; i < N - 1; i++) {
    int min = i;

    for (int j = i + 1; j < N; j++)
      if (A[j] < A[min]) min = j;

    swap(A[i], A[min]);
  }
}
```



Insertion Sort



- Concept: Method used by card players to sort cards in hand; considers one element at a time and inserts it into its correct position among already sorted elements.
- Complexity: $O(N^2)$ steps on average.

Insertion Sort: Implementation




```
void insert_sort(int a[], int size) {
  for (int i = 1; i < size; i++) {
    int v = a[i]; // Current element to insert
    int j = i;

    while (j > 0 && v < a[j - 1]) {
      a[j] = a[j - 1];
      j--;
    }

    a[j] = v;
  }
}
```

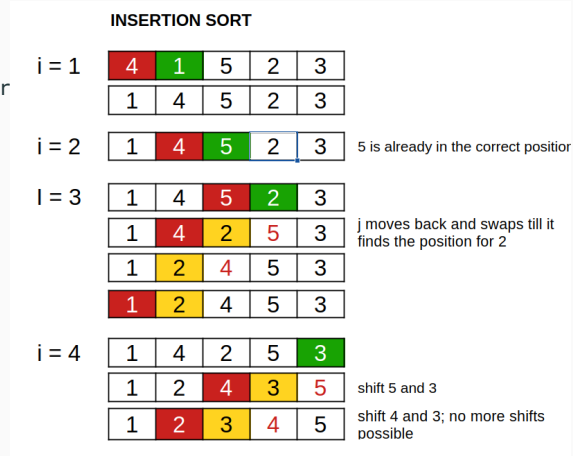
- the for loop ensures we are inserting the i-th element in the correct position (which is in a position left of `i`): hence we start from the second element (index 1)
- `v` holds the element to shift
- the while loop does the job of shifting all elements before the i-th one step to the right, to make it possible to insert the i-th element where it belongs

Insertion Sort: Example

```
void insert_sort(int a[], int size) {
  for (int i = 1; i < size; i++) {
    int v = a[i]; // Current element to inser
    int j = i;

    while (j > 0 && v < a[j - 1]) {
      a[j] = a[j - 1];
      j--;
    }

    a[j] = v;
  }
}
```



Bubble Sort



- Concept: Based on exchanges of adjacent elements if necessary until no more swaps are required.
- Complexity: $O(N^2)$ steps on average.

Bubble Sort: Implementation




```
void bubble_sort(int a[], int size) {
  bool swaps = true;

  while (swaps) {
    swaps = false;

    for (int i = size - 1; i > 0; i--) {
      if (a[i - 1] > a[i] ) {
        swaps = true;
        swap(a[i - 1], a[i]);
      }
    }
  }
}
```

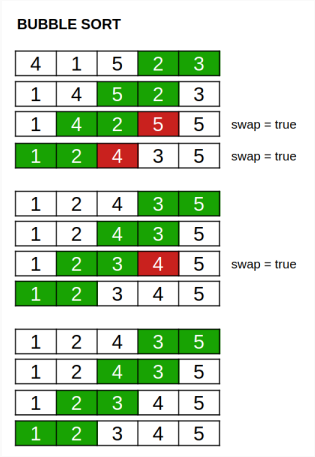
- start from the right (arbitrary choice)
- compare adjacent elements, if there is a swap, remember there has been and continue
- repeat till there is a pass with no swaps (you have verified the array is sorted)
- intuitively, each for loop moves lower elements closer to the array start (although it might require multiple passes to bring the elments in their correct position)

Bubble Sort: Example

```
void bubble_sort(int a[], int size) {
  bool swaps = true;

  while (swaps) {
    swaps = false;

    for (int i = size - 1; i > 0; i--) {
      if (a[i - 1] > a[i] ) {
        swaps = true;
        swap(a[i - 1], a[i]);
      }
    }
  }
}
```



Comparison



In general, all three sorting functions are $O(N^2)$

- However, one can look at best and worst case as well:
  - if data is nearly sorted, insertion sort and bubble sort are more efficient than selection sort (they are $O(N)$)
  - if writing costs more than comparison, selection sort is better
- Selection sort is probably the simplest to understand
- There are more efficient algos => for instance quicksort

QuickSort

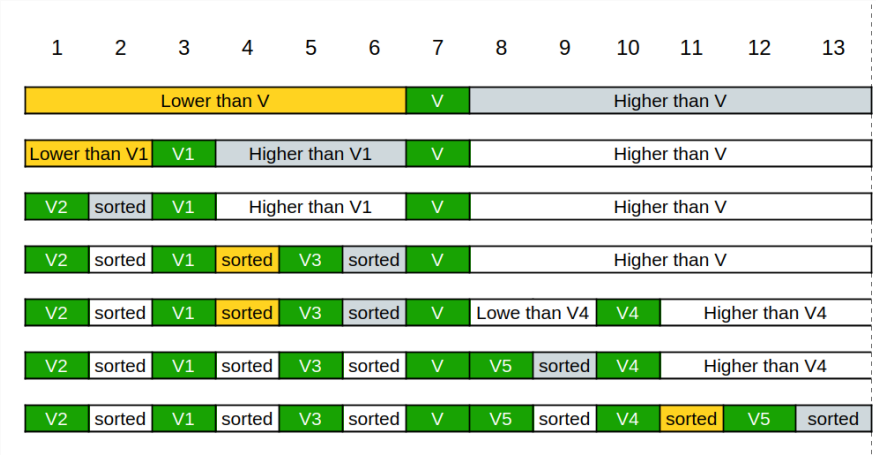


- Concept: A divide-and-conquer sorting algorithm based on array partitioning around a pivot.
- Partitioning: We make sure that:
  - For some value of i, the element A[i] is in the right place
  - All elements A[0], ..., A[i-1] are less than or equal to A[i]
  - All elements A[i + 1], ..., A[N-1] are greater than or equal to A[i]
- Steps:
  - The array is sorted by partitioning and recursively applying the method to the sub-arrays.
- Complexity: $O(N \log_2(N))$ steps on average.



- Pivot selection: we arbitrarily choose an element (`A[v]` called pivot or partitioning element) usually, the leftmost or the rightmost element is selected
- Partitioning:
  - The partitioning is made in place: we keep modifying the same array
  - The array is partitioned so that:
    - All elements for which `A[i]` $\leq$ `A[v]` are positioned before the pivot
    - All elements for which `A[j]` $\geq$ `A[v]` are positioned after the pivot
  - After this operation, the pivot is in its final position
  - Once we have partitioned the array, we apply the algo recursively to the left and right partition

Quicksort: Explanation



Swap




```
// old fashioned C
void swap(int *a, int *b){
  int t = *a;
  *a = *b;
  *b = t;
}
```

Quicksort: Algorithm




```
int partition(int A[], int start, int end) {
  int pIndex = start, v = A[end];

  for (int i = start; i < end; i++){
    if (A[i] <= v){
      swap(A[i], A[pIndex]);
      pIndex++;
    }
  }

  swap(A[pIndex], v);

  return pIndex;
}
```

- I choose last element as pivot (`v`)
- I then loop over the array: when an element is lower than the Pivot, I move it to a lower position and increase `pIndex`
- At the end of the loop, `pIndex` will point to the position left of which all elements are lower than `v`
- I can now place `v` in the position pointed by `pIndex`
- Notice that left partition and right partition are not sorted

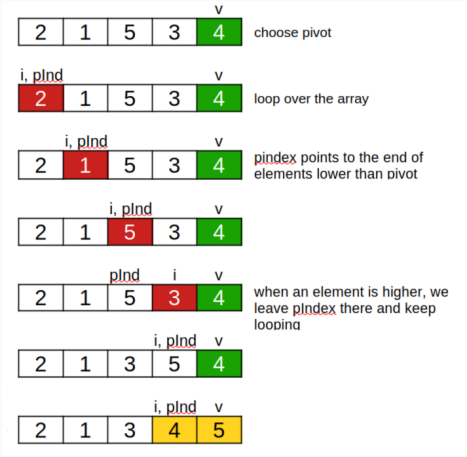
Quicksort: partitioning example

```
int partition(int A[], int start, int end) {
  int pIndex = start, v = A[end];

  for (int i = start; i < end; i++){
    if (A[i] <= v){
      swap(A[i], A[pIndex]);
      pIndex++;
    }
  }

  swap(A[pIndex], v);

  return pIndex;
}
```



Quicksort: part II

```
// nice entry point
void quicksort(int A[], int N) {
  quicksort_aux(A, 0, N-1);
}

// add args required for recursion
void quicksort_aux(int A[], int start, int end) {
  if (end <= start) return;

  int pivot = partition(A, start, end);

  quicksort_aux(A, start, pivot-1);
  quicksort_aux(A, pivot+1, end);
}
```

- this is just the ”glue” to recursively apply `quicksort`

Shell Sort



- Concept: Improves Insertion sort by allowing exchanges between elements at a distance h, reducing the problem of slow contiguous shifts.
- Complexity: $O(N^{3/2})$ steps on average for the standard implementation.

Other operations on sorted arrays




```
Once we have a sorted array, it is a good idea to keep it sorted, even
when we have to modify it.
```

- Checking whether an array is sorted (forward or reverse)
- Insertion of an element
- Merging two sorted arrays
- Deleting an element

Checking whether an array is sorted



- Check in $O(N)$ steps

```
bool is_sorted(const int a[], int size) {
  bool sorted = true;
  for (int i = 1; i < size; i++) {
    sorted = sorted && (a[i - 1] <= a[i]);
  }
  return sorted;
}
```

Checking whether an array is sorted (v2)



- Check in $O(N)$ steps
- Why better than the previous version?

```
bool is_sorted(const int a[], int size) {
  for (int i = 1; i < size; i++) {
    if (a[i - 1] > a[i]) return false;
  }
  return true;
}
```

Insertion in an ordered array



- Insert an element into an ordered array in $O(N)$ steps.
- E.g., `insert(5, [1 3 4 8])` → `[1 2 3 4 5 8]`

`./cpp/array_insert.cpp`

Insertion on Ordered Arrays: Comments



- On which values for `element` should I test my procedure?
- Implementation with one variable is messier
- This will not work (because after inserting the element, the index where insertion has to happen in b is that of a plus 1)

```
int i = 0;
while (a[i] < element) b[i] = a[i++];
b[i++] = element;
while (i < size) b[i] = a[i++];
```

Ordered Merge



- Combine two sorted arrays in $O(N_1 + N_2)$ steps.
- E.g., `merge([1 3 4 8], [2 3 5 6])` → `[1 2 3 4 5 6 8]`

Ordered Merge: General Structure




```
#include <iostream>
using namespace std;

void merge(int result[], int a[], int a_size, int b[], int b_size) {
  // ...
}
void print(int a[], int size) {
  for (int i = 0; i < size; i++) cout << a[i] << " ";
}

int main() {
  int a[6] = { 5, 6, 12, 34, 88, 106 };
  int b[3] = { 45, 56, 67 };
  int result[6 + 3];

  merge(result, a, 6, b, 3);
  print(b, SIZE);
}
```

Ordered Merge: Approach



1. Think about nominal Algo: loop over a and b, check which element is lower, put it in result, increment the index of the array whose element you have selected and continue till you have all elements
2. Think about corner cases: Will nee to consider:
   - when all elements of a come before b (and viceversa, of course)
   - when I finish inserting all elements of a and there are still elements in b

Ordered Merge: Implementation




```
./cpp/ordered_merge.cpp
```

```
5 6 12 34 45 56 67 88 106
```

Deletion



- Delete first occurrence only (delete is a reserved keyword)
- Return new size

```
int delete_eleement(int a[], int size, int element) {
  int index = find(a, size, element);

  if (index == -1) return size;

  for (int i = index; i < size - 1; i++) {
    a[i] = a[i + 1];
  }

  return size - 1;
}
```

Deletion, multiple occurrences



- Delete all occurrences

```
int delete_all(int a[], int size, int element) {
  int index = 0;
  int deleted = 0;

  while (find(a, size, element) != -1) {
    index = find(a, size, element);  // inefficient
    for (int i = index; i < (size - deleted) - 1; i++) {
      a[i] = a[i + 1];
    }
    deleted ++;
  }

  return size - deleted;
}
```

Deletion, multiple occurrences (v2)



- Delete all occurrences
- No need to find each element twice

```
int delete_all(int a[], int size, int element) {
  int index = 0;
  int deleted = 0;

  while ((index = find(a, size, element)) != -1) {
    for (int i = index; i < (size - deleted) - 1; i++) {
      a[i] = a[i + 1];
    }
    deleted ++;
  }

  return size - deleted;
}
```

Some Golden Rules



By Donald Knuth:

1. Premature optimization is the root of all evils
2. Get it working correctly first, then make it fast, then make it pretty.